

ISEN/DAEN 427 Lecture 7

Alexander Abuabara

Bayesian Linear Regression Models

Bayesian Modeling and Computation in Python, Ch. 3



From Linear Regression to Bayesian Linear Regression

Classical Linear Regression

$$y_i = X_i\beta + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

Goal: Find the single “best” estimate

$$\hat{\beta} = \arg \min_{\beta} \sum_i (y_i - X_i\beta)^2$$

→ Parameters are fixed, estimated by least squares.

Bayesian Linear Regression

$$y_i \sim \mathcal{N}(X_i\beta, \sigma^2)$$

We place priors:

$$\beta \sim \mathcal{N}(0, \tau^2)$$

$$\sigma \sim \text{Half-StudentT}(\nu, s)$$

→ Parameters are random variables with uncertainty.

Classical regression finds one estimate.

Bayesian regression infers a *distribution* over possible models.

From Classical to Bayesian Linear Regression

Frequentist Linear Regression

$$y_i = X_i\beta + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

Parameters β, σ are fixed but unknown.

$$\hat{\beta} = \arg \min_{\beta} \sum_i (y_i - X_i\beta)^2$$

We find point estimates (OLS or MLE).

Bayesian Linear Regression

$$y_i \sim \mathcal{N}(X_i\beta, \sigma^2)$$

Place priors on parameters:

$$\beta \sim \mathcal{N}(0, \tau^2)$$

$$\sigma \sim \text{Half-StudentT}(\nu, s)$$

Inference: compute posterior

$$p(\beta, \sigma | y) \propto p(y | \beta, \sigma) p(\beta) p(\sigma)$$

Both use the same likelihood: Bayesian adds priors to express uncertainty in parameters.

Example: Predicting House Prices

Model Setup

We assume each observed price y_i follows

$$y_i = X_i \beta + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

→ *Price = predicted value + random noise.*

Features and Parameters

Suppose we predict house price using:

$$X_i = \begin{bmatrix} \text{size}_i \\ \text{bedrooms}_i \\ \text{age}_i \end{bmatrix}, \quad \beta = \begin{bmatrix} \beta_{\text{size}} \\ \beta_{\text{bedrooms}} \\ \beta_{\text{age}} \end{bmatrix} \in \mathbb{R}^3$$

The model learns how each feature influences price through β .

Linear Regression Intuition

1. Likelihood of Each Observation

We assume that each outcome y_i is normally distributed around its predicted value:

$$\mathbb{P}(y_i \mid s) = \mathcal{N}(X_i\beta, \sigma^2)$$

→ Think: predictions $X_i\beta$ plus random noise with spread σ .

2. Equivalent Model Representation

$$y_i = X_i\beta + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

→ The error term ϵ_i captures what the model can't explain.

3. Parameters and Inputs

- $\beta \in \mathbb{R}^p$: weights linking features to outcomes $\rightarrow$ *How much each feature contributes to the prediction.*
- $X_i^\top \in \mathbb{R}^p$: feature vector for observation $i \rightarrow$ *The observed inputs (e.g., size, age, price).*
- σ : standard deviation of noise $\rightarrow$ *How uncertain or “spread out” our predictions are.*

Linear regression assumes data = deterministic signal + random noise.

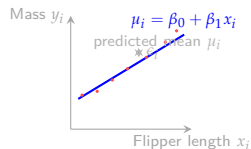
Bayesian Linear Regression: Adelie Penguin Example

Model Specification

The model assumes a linear relationship between flipper length (x_i) and body mass (y_i):

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

→ *Body mass = intercept + slope × flipper length + noise.*



Observed masses scatter around the regression line.

Priors

$$\beta_0 \sim \mathcal{N}(0, 4000^2) \quad (\text{intercept})$$

$$\beta_1 \sim \mathcal{N}(0, 4000^2) \quad (\text{slope})$$

$$\sigma \sim \text{Half-StudentT}(\nu = 100, \text{scale} = 2000) \quad (\text{noise scale})$$

→ *Wide priors: we start with little prior knowledge.*

PyMC samples the posterior: $p(\beta_0, \beta_1, \sigma \mid \text{data})$.

Using PyMC

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import pymc as pm
import arviz as az

penguins = pd.read_csv("penguins.csv")
penguins = penguins.dropna(axis=0, how='any', inplace=False)
penguins.head()

adelie_mask = (penguins["species"] == "Adelie")
adelie_mass_obs = penguins.loc[adelie_mask, "body_mass_g"].values
adelie_flipper_length_obs = penguins.loc[adelie_mask, "flipper_length_mm"]
```



Figure 1: Data on different penguin species.

```
with pm.Model() as model_adelie_flipper_regression:
    adelie_flipper_length = pm.Data("adelie_flipper_length",
                                     adelie_flipper_length_obs)
    sigma = pm.HalfStudentT("sigma", 100, 2000)
    beta_0 = pm.Normal("beta_0", 0, 4000)
    beta_1 = pm.Normal("beta_1", 0, 4000)
    mu = pm.Deterministic("mu", beta_0 + beta_1 * adelie_flipper_length)

    mass = pm.Normal("mass", mu=mu, sigma=sigma, observed=adelie_mass_obs)

    inf_data_adelie_flipper_regression = pm.sample(return_inferencedata=True)

graphviz = pm.model_to_graphviz(model_adelie_flipper_regression)
graphviz
```

Using PyMC

```
axes = az.plot_posterior(inf_data_adelie_flipper_regression,  
                        var_names=["beta_0", "beta_1"], fontsize=20)  
plt.savefig("adelie_coefficient_posterior_plots")
```

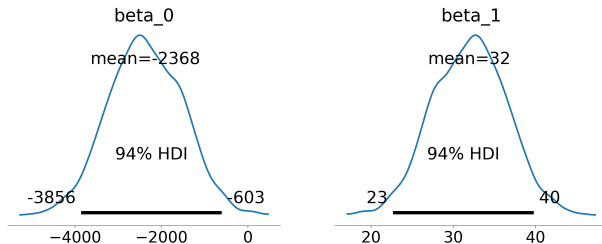


Figure 2: Posterior Plots

```
beta0_m = inf_data_adelie_flipper_regression.posterior.mean().to_dict()["data_vars"]["beta_0"]["data"]
beta1_m = inf_data_adelie_flipper_regression.posterior.mean().to_dict()["data_vars"]["beta_1"]["data"]

flipper_length = np.linspace(adelie_flipper_length_obs.min(),
                             adelie_flipper_length_obs.max(), 100)

flipper_length_mean = beta0_m + beta1_m * flipper_length

fig, ax = plt.subplots()
ax.plot(flipper_length, flipper_length_mean, c='C4',
        label=f'y = {beta0_m:.2f} + {beta1_m:.2f} * x')
ax.scatter(adelie_flipper_length_obs, adelie_mass_obs)
ax.plot_hdi(adelie_flipper_length_obs,
            inf_data_adelie_flipper_regression.posterior['mu'],
            hdi_prob=0.94, color='r', ax=ax)
ax.set_xlabel('Flipper Length')
ax.set_ylabel('Mass')
```

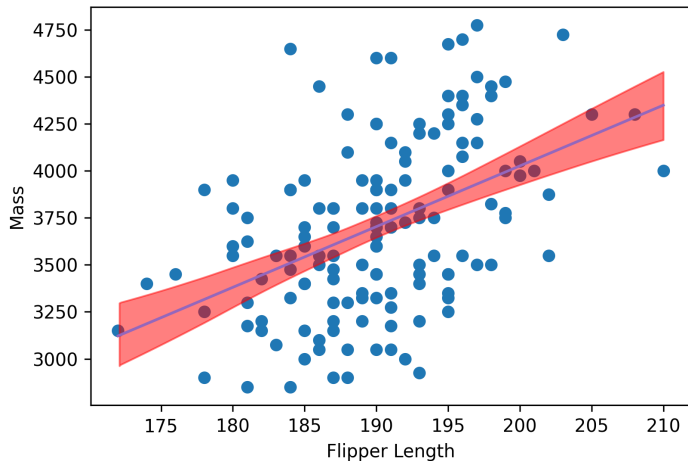


Figure 3: Bayesian Linear Regression Model

Logistic Regression

Logistic regression is used to model the probability of a binary outcome $y_i \in \{0, 1\}$ given a vector of features $X_i \in \mathbb{R}^p$.

We denote the model parameters (weights) as $\beta \in \mathbb{R}^p$.

- Logistic regression predicts probabilities, not just 0 or 1.
- The log-odds of the event $y_i = 1$ are linear in the features X_i .
- The sigmoid function converts linear combinations $X_i^\top \beta$ into probabilities.

Logistic Regression: Log-odds (Logit) Function

The model assumes that the **log-odds** of the probability is a linear function of the inputs:

$$\log \frac{\mathbb{P}(y_i = 1 \mid \beta)}{\mathbb{P}(y_i = 0 \mid \beta)} = X_i^\top \beta$$

This means that the logarithm of the odds of the event $y_i = 1$ is proportional to a linear combination of the features.

Logistic Regression: Probability Formulation

Rearranging the log-odds equation, we can express the probability directly as:

$$\mathbb{P}(y_i = 1 \mid \beta) = \frac{1}{1 + \exp(-X_i^\top \beta)}$$

and equivalently,

$$\mathbb{P}(y_i = 0 \mid \beta) = \frac{1}{1 + \exp(X_i^\top \beta)}$$

Logistic Regression: Sigmoid Function

The function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

is called the **sigmoid function**.

It maps any real-valued number to the interval $(0, 1)$.

Logistic Regression

With $s := \beta \in \mathbb{R}^p$, the likelihood of observation $y_i \in \{0, 1\}$ is:

$$\log \frac{\mathbb{P}(y_i = 1 | \beta)}{\mathbb{P}(y_i = 0 | \beta)} = X_i \cdot \beta$$

or equivalently

$$\mathbb{P}(y_i = 1 | \beta) = \frac{1}{1 + \exp(-X_i \cdot \beta)} \quad \text{sigmoid function}$$

$$\mathbb{P}(y_i = 0 | \beta) = \frac{1}{1 + \exp(X_i \cdot \beta)}$$

where $X_i^\top \in \mathbb{R}^p$.

Using PyMC

```
species_filter = penguins["species"].isin(["Adelie", "Chinstrap"])
bill_length_obs = penguins.loc[species_filter, "bill_length_mm"].values
species = pd.Categorical(penguins.loc[species_filter, "species"])
```

Using PyMC

```
with pm.Model() as model_logistic_penguins_bill_length:
    beta_0 = pm.Normal("beta_0", mu=0, sigma=10)
    beta_1 = pm.Normal("beta_1", mu=0, sigma=10)

    mu = beta_0 + pm.math.dot(bill_length_obs, beta_1)
    theta = pm.Deterministic("theta", pm.math.sigmoid(mu))

    y1 = pm.Bernoulli("y1", p=theta, observed=species.codes)

    bd = pm.Deterministic("bd", -beta_0/beta_1)
    idata_logistic_penguins_bill_length = pm.sample(
        5000, chains=2, idata_kwargs={"log_likelihood": True})
    idata_logistic_penguins_bill_length.extend(
        pm.sample_prior_predictive(samples=10000))
    idata_logistic_penguins_bill_length.extend(
        pm.sample_posterior_predictive(idata_logistic_penguins_bill_length))

graphviz = pm.model_to_graphviz(model_logistic_penguins_bill_length)
graphviz
```

```
colors = ['blue', 'red']

fig, ax = plt.subplots(figsize=(6, 4))
plt.ylim(-0.075, 1.075)
theta = idata_logistic_penguins_bill_length.posterior['theta'].mean(('chain', 'draw'))
idx = np.argsort(bill_length_obs)
ax.vlines(idata_logistic_penguins_bill_length.posterior['bd'].values.mean(), 0, 1, color='black')
bd_hpd = az.hdi(idata_logistic_penguins_bill_length.posterior['bd'].values.flatten(), ax=ax)
plt.fill_betweenx([0, 1], bd_hpd[0], bd_hpd[1], color='green', alpha=.2, label='decision boundary')
for i, (label, marker) in enumerate(zip(species.categories, ('.', 's'))):
    _filter = (species.codes == i)
    x = bill_length_obs[_filter]
    np.random.seed(42)
    y = np.random.normal(i, 0.02, size=_filter.sum())
    ax.scatter(bill_length_obs[_filter], y, label=label, alpha=.5, s = 20, color=colors[i])
ax.plot(bill_length_obs[idx], theta[idx], color='purple', zorder=10, label='logistic model')
az.plot_hdi(bill_length_obs, idata_logistic_penguins_bill_length.posterior['theta'].values,
            color='C4', ax=ax, plot_kwargs={'zorder':10})
ax.set_xlabel('Bill Length (mm)')
ax.set_ylabel(' ', rotation=0)
ax.legend(loc=5)
```

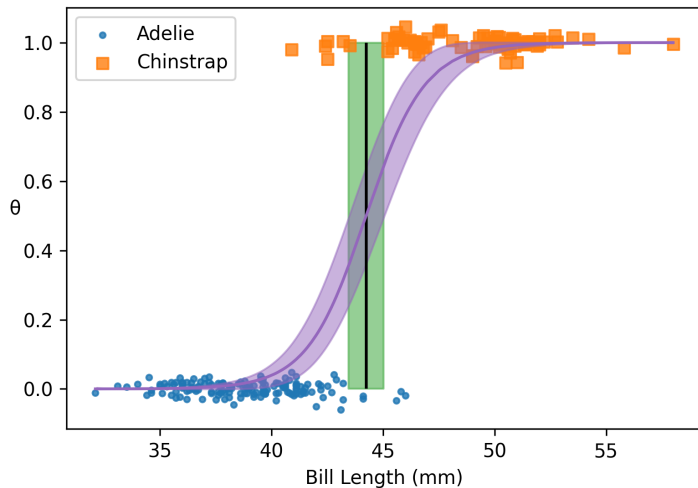


Figure 4: Bayesian Logistic Regression Model